# CPSC 4420/6420: Artificial Intelligence — Assignment 2

**Name:** _____ **Environment:** Gymnasium Taxi-v4 **Method:** tabular Q-learning

**Code repository:** https://github.com// (code, README with setup/run instructions, all plots, metrics.json and the MP4 video `results/taxi_agent.mp4`).

All learning curves are averaged over **5 independent seeds** (shaded band = ±1 std across seeds, curves smoothed with a 50-episode moving average). Every "greedy eval" number is the policy run with exploration disabled (ε = 0) for **100 episodes** per seed.

## (A) Environment description

**Goal.** Taxi (Dietterich, 2000) is a 5×5 grid world with four marked locations R(0,0), G(0,4), Y(4,0) and B(4,3). A passenger waits at one of the four locations and wants to go to a different one. The taxi must drive to the passenger, pick them up, drive to the destination and drop them off, in as few steps as possible. Internal walls block some east/west moves.

**State / observation space.** Discrete(500). The state encodes (taxi row, taxi column, passenger location, destination) as ((row·5 + col)·5 + passenger)·4 + destination, where passenger ∈ {R, G, Y, B, in-taxi} and destination ∈ {R, G, Y, B}: 25 × 5 × 4 = 500 states. Episodes start uniformly in one of 300 states (passenger not in the taxi and not at its destination). Because the space is already discrete and small, **no discretization is needed**. The Q-table is 500 × 6.

**Action space.** Discrete(6): 0 = south, 1 = north, 2 = east, 3 = west, 4 = pickup, 5 = drop-off.

**Transition dynamics.** Deterministic under the default settings used here (`is_rainy=False`, `fickle_passenger=False`). Moves change the taxi position by one cell; moving into a wall or the border leaves the taxi in place. Pickup succeeds only when the taxi is on the passenger's cell; drop-off succeeds at the destination (dropping the passenger at another marked location just moves them there). The only randomness is the initial state.

**Reward structure.** −1 per step; +20 for delivering the passenger at the destination; −10 for an illegal pickup or drop-off. A shortest delivery takes roughly 8–16 steps, so the optimal episode reward is about +5 to +15 (≈ +8 on average).

**Termination.** The episode *terminates* when the passenger is dropped at the destination. It is *truncated* by the TimeLimit wrapper after 200 steps (so the worst possible return is about −2000 if every action were an illegal pickup, and −200 for an aimless wander). **Note:** current Gymnasium (1.3+) registers this environment as `Taxi-v4`; v4 only fixed the optional rainy/fickle variants and is identical to Taxi-v3 with default settings.

## (B) Q-learning implementation

The agent is in `qlearning.py` and is written from scratch (no RL libraries other than Gymnasium for the environment):

• **Hyperparameters** live in one dataclass, `QConfig(alpha, gamma, epsilon_start, epsilon_min, epsilon_decay, episodes, q_init, max_steps)`, so every experiment is a one-line `replace(BASELINE, alpha=0.5)`.

• **Q-value initialization:** a 500 × 6 NumPy array filled with `q_init` (0 by default).

• **ε-greedy policy:** with probability ε pick a uniformly random action, otherwise $\text{argmax}_a$ Q(s,a) with random tie-breaking (important early on when a whole row is 0).

• **Exploration schedule:** ε starts at $\varepsilon_{start}$ and is multiplied by ε-decay after every episode, floored at $\varepsilon_{min}$.

• **Update:** $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$.

• **Terminal-state handling:** when the step *terminates* (passenger delivered) the bootstrap term is dropped, i.e. the target is just r. When the episode is only *truncated* by the 200-step limit, s' is not terminal, so we still bootstrap from max Q(s',·). Treating truncation as termination would wrongly teach the agent that some states are worth 0.

• **Training loop:** reset → (act, step, update) until terminated/truncated → record reward, length and success → decay ε. **Evaluation:** pure greedy policy, ε = 0, separate seeds.

Core update (verbatim from the code):

```
future = 0.0 if terminated else gamma * max(Q[s_next])
Q[s, a] += alpha * (r + future - Q[s, a])
```

## (C) Baseline training

**Baseline:** $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon_{start} = 1.0$, ε-decay = 0.995 per episode, $\varepsilon_{min} = 0.01$, 2000 episodes, Q initialized to 0. With this decay, ε reaches its floor of 0.01 after about 920 episodes. These are standard textbook defaults: a conservative learning rate, a far-sighted discount, and fully random exploration at first.

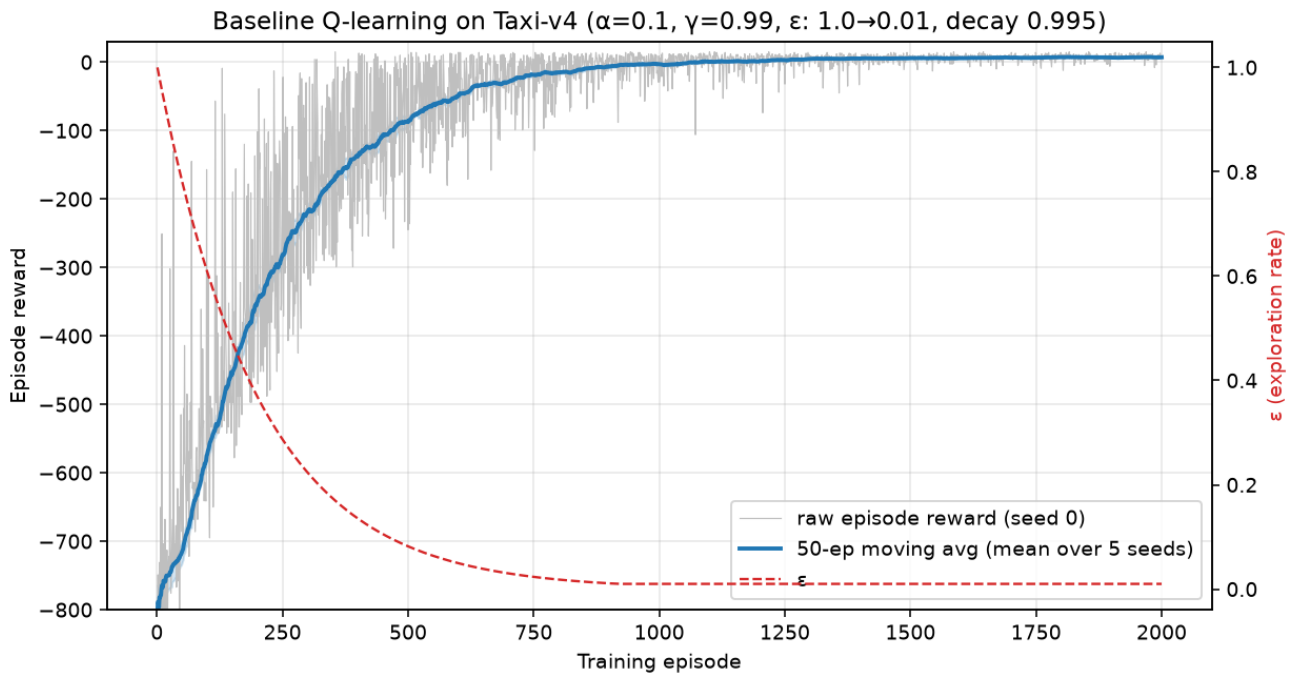



Figure 1 – Baseline training reward per episode (grey: one raw seed; blue: moving average over seeds; red dashed: ε schedule).

**Initial results.** The first episodes are almost random walks that hit the 200-step limit with many illegal pickups/drop-offs (reward ≈ −700 to −800). As Q-values propagate back from the +20 delivery and ε falls, reward climbs steadily. The 50-episode moving average first reaches 0 at episode **1071**, and the last 200 training episodes average **6.94**, close to optimal. However, the greedy evaluation from random start states gives only **-21.33** mean reward with a **85.6%** success rate. With α = 0.1, Q-values move slowly, and after 2000 episodes some rarely visited state–action pairs still have values that make the greedy policy loop (for example between two cells) until the 200-step limit. So the baseline *looks* converged on the training curve but is not yet reliable. That motivates the sweeps below.

## (D) Learning-rate experiment

α ∈ {0.01, 0.1, 0.5, 0.9}; everything else fixed at the baseline.

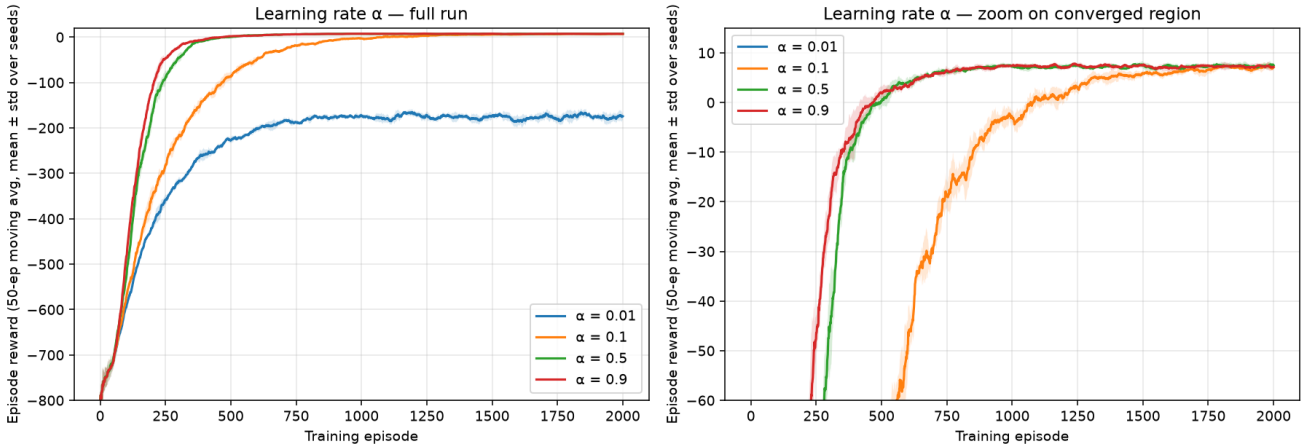



Figure 2 – Effect of the learning rate α (left: full run; right: zoom near the optimum).

| α | Episodes until MA reward ≥ 0 | Mean train reward (all eps) | Train reward last 200 (± std) | Greedy eval reward | Eval success rate | Eval steps |
|---|---|---|---|---|---|---|
| α = 0.01 | never | -232.5 | -174.95 ± 89.38 | -385.69 | 0.2% | 199.6 |
| α = 0.1 | 1071 | -80.8 | 6.94 ± 4.13 | -21.33 | 85.6% | 39.3 |
| α = 0.5 | 488 | -41.4 | 7.40 ± 3.36 | 7.70 | 99.8% | 13.3 |
| α = 0.9 | 467 | -36.1 | 7.26 ± 3.43 | 8.13 | 100.0% | 12.9 |

**Learning speed.** Larger α learns much faster: α = 0.9 reaches a non-negative moving average after 467 episodes and α = 0.5 after 488, versus 1071 for α = 0.1. α = 0.01 never gets there in 2000 episodes (final training reward ≈ -175, greedy success 0.2%). Each update moves Q only 1% toward the target, so information from the +20 reward barely propagates back along the ~10-step paths.

**Stability.** Usually a high α is unstable because it chases noisy targets. Taxi is *deterministic*, though: every (s,a) always gives the same r and s', so the TD target has no sampling noise, and α close to 1 just copies the (correct) target. The across-seed bands for α = 0.5 and 0.9 are narrow, and their last-200-episode standard deviations (3.36, 3.43) are no larger than α = 0.1's. The remaining variance comes from the random start state and the 1% residual exploration, not from the learning rate. (Part H shows this reverses once the environment becomes stochastic.)

**Final performance.** α = 0.5 and α = 0.9 give near-optimal greedy policies (7.70 and 8.13 mean reward, 99.8% / 100.0% success, about 13 steps). α = 0.1 is still under-trained (-21.33), and α = 0.01 has failed.

## (E) Discount-factor experiment

γ ∈ {0.5, 0.9, 0.99}; everything else fixed at the baseline (α = 0.1).

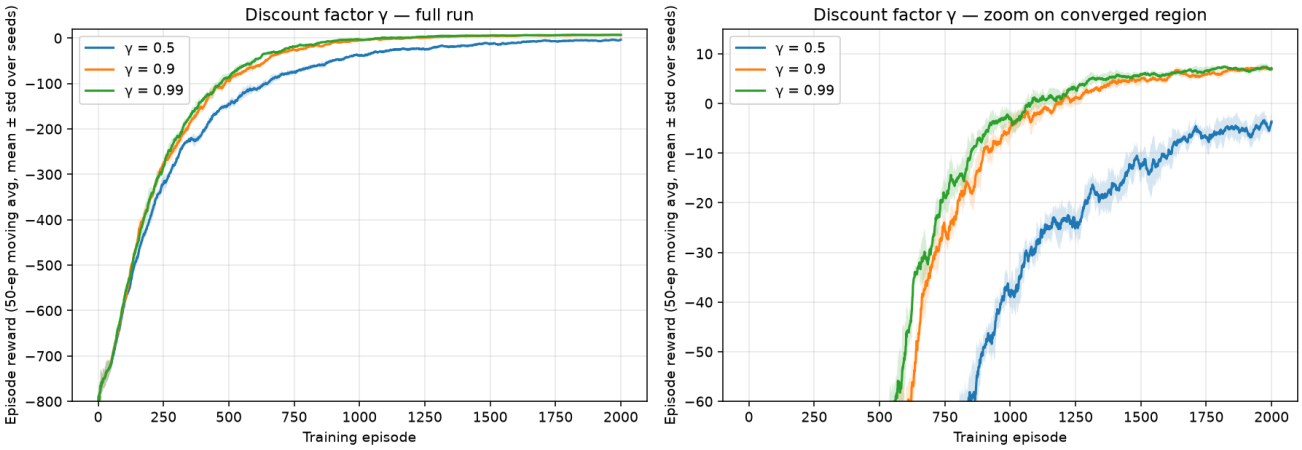



Figure 3 – Effect of the discount factor γ.

| γ | Episodes until MA reward ≥ 0 | Mean train reward (all eps) | Train reward last 200 (± std) | Greedy eval reward | Eval success rate | Eval steps |
|---|---|---|---|---|---|---|
| γ = 0.5 | never | -113.5 | -4.70 ± 16.83 | -135.02 | 30.8% | 141.5 |
| γ = 0.9 | 1177 | -84.4 | 6.92 ± 4.35 | -37.28 | 79.6% | 50.4 |
| γ = 0.99 | 1071 | -80.8 | 6.94 ± 4.13 | -21.33 | 85.6% | 39.3 |

**Immediate vs. future reward.** The only positive reward (+20) comes at the end of a multi-step plan, while every intermediate step costs −1. With γ = 0.5, the delivery reward seen k steps ahead is worth $20 \cdot 0.5^k$: only 0.6 at k = 5 and 0.02 at k = 10. Meanwhile the discounted cost of any infinite wander is bounded by $-1/(1-\gamma) = -2$. Far from the goal, all four movement actions therefore have almost identical Q-values ($\approx -2$). The agent is essentially myopic and gets no useful signal about which way to drive. Its greedy policy delivers in only 30.8% of evaluation episodes (mean reward -135.02), and its training curve never reaches 0.

γ = 0.9 has a 10-step effective horizon, which is about one Taxi trip, so it learns a usable policy (79.6% success). It is still worse than γ = 0.99 (85.6%, eval reward -21.33 vs -37.28) because value differences between good and bad routes from distant states stay small and are easily swamped by the not-yet-converged parts of the table. A discount close to 1 matches the task: the agent should care about the whole trip, and the −1 step cost already rewards short paths, so no extra discounting is needed for that.

## (F) Additional hyperparameter: ε-decay rate

ε-decay ∈ {0.95, 0.99, 0.995, 0.999} per episode (ε reaches 0.01 after about 90, 460, 920 and 4600 episodes respectively), all other settings at the baseline.

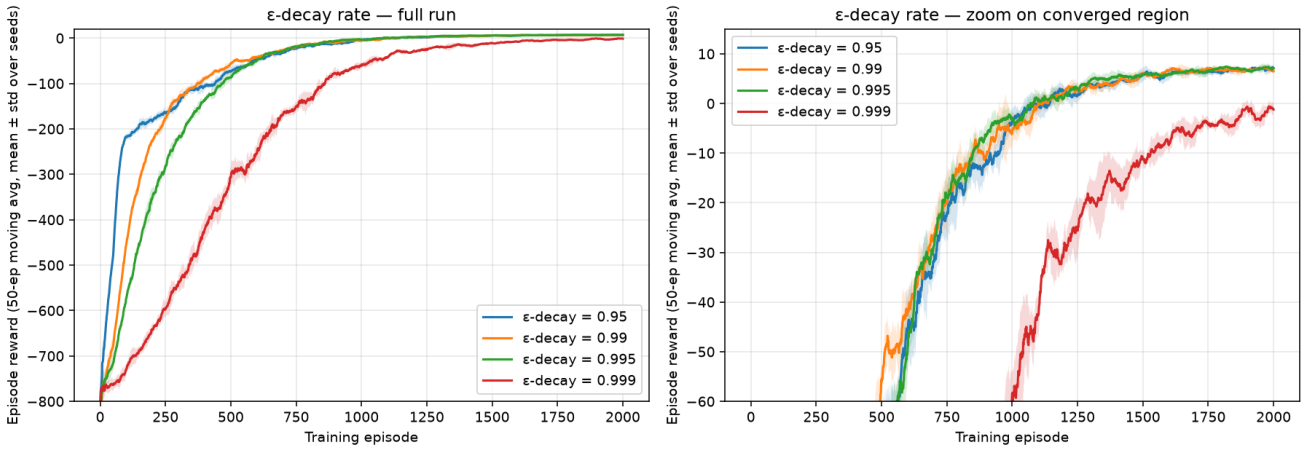



Figure 4 – Effect of the ε-decay rate (exploration schedule).

| ε-decay | Episodes until MA reward ≥ 0 | Mean train reward (all eps) | Train reward last 200 (± std) | Greedy eval reward | Eval success rate | Eval steps |
|---|---|---|---|---|---|---|
| ε-decay = 0.95 | 1088 | -48.1 | 6.93 ± 4.61 | -21.06 | 91.0% | 29.4 |
| ε-decay = 0.99 | 1100 | -60.8 | 6.75 ± 4.55 | -40.86 | 86.6% | 37.5 |
| ε-decay = 0.995 | 1071 | -80.8 | 6.94 ± 4.13 | -21.33 | 85.6% | 39.3 |
| ε-decay = 0.999 | never | -184.0 | -2.40 ± 11.83 | -14.31 | 89.0% | 33.0 |

**Effect on learning.** This is the exploration/exploitation trade-off. Fast decay (0.95) stops exploring after about 90 episodes and has the best average training reward (-48.1), since fewer random actions are wasted. But its greedy policy varies a lot between seeds (see the per-seed values in metrics.json), because some states were never explored enough. Slow decay (0.999) keeps ε above 0.13 for the whole 2000-episode run, so the training reward stays low (-2.40 over the last 200 episodes, with high variance) and its moving average never reaches 0. Its greedy policy is the best of the four, though (-14.31, 89.0% success), because broad exploration visits more of the 300 start states. With $\alpha = 0.1$ none of the schedules is fully converged in 2000 episodes; the learning rate, not the exploration schedule, is the bottleneck for this budget.

## (G) Best agent

**Selection procedure.** (1) From each single-parameter sweep (D, E, F) take the value with the highest greedy evaluation reward averaged over seeds. Evaluation-reward differences smaller than 0.5 are treated as ties (that is within evaluation noise), and ties go to the setting that learned fastest (highest mean training reward over all episodes). The winners were $\alpha = 0.9$, $\gamma = 0.99$, ε-decay = 0.999. (2) Re-train candidate combinations (baseline, all three combined, best α only, best ε-decay only) and apply the same rule.

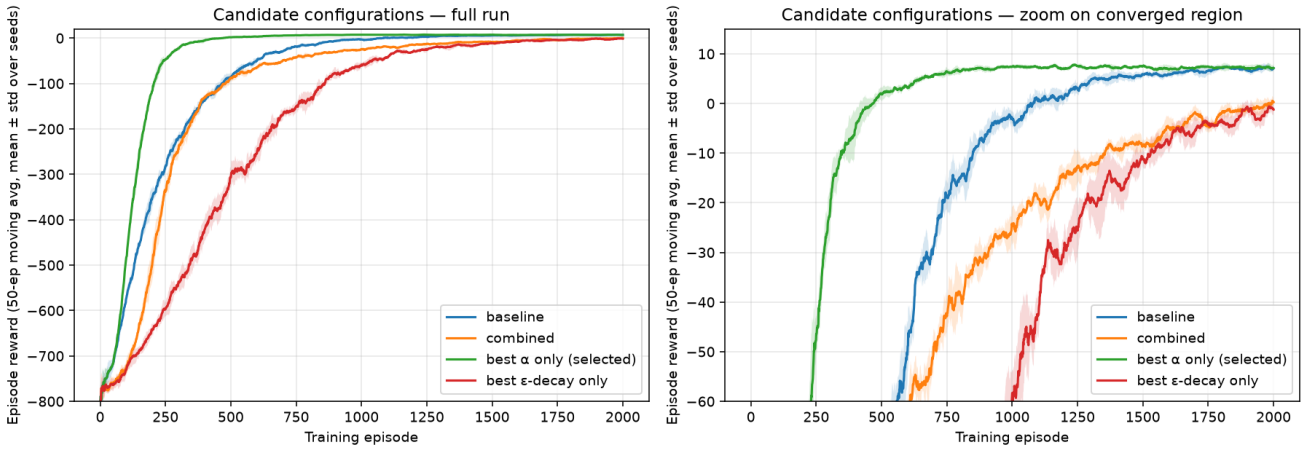



Figure 5 – Candidate configurations for the best agent.

| Candidate | Episodes until MA reward ≥ 0 | Mean train reward (all eps) | Train reward last 200 (± std) | Greedy eval reward | Eval success rate | Eval steps |
|---|---|---|---|---|---|---|
| baseline | 1071 | -80.8 | 6.94 ± 4.13 | -21.33 | 85.6% | 39.3 |
| combined | 1980 | -107.8 | -0.83 ± 9.90 | 8.20 | 100.0% | 12.8 |
| best α only | 467 | -36.1 | 7.26 ± 3.43 | 8.13 | 100.0% | 12.9 |
| best ε-decay only | never | -184.0 | -2.40 ± 11.83 | -14.31 | 89.0% | 33.0 |

"Combined" and "best α only" tie at the optimum in greedy reward, but "combined" inherits the slow 0.999 exploration schedule and needs about 4× as many episodes to learn. The **selected agent is "best α only"**: α = 0.9, γ = 0.99, ε: 1.0 → 0.01 with decay 0.995, 2000 episodes.

**Final evaluation** (seed-0 agents, the same 100 start states for every policy, ε = 0). As a reference upper bound, the table also shows the true optimal policy computed by value iteration on the known Taxi model (not used for training).

| Policy | Mean reward ± std | Success rate | Mean steps | Illegal actions / ep |
|---|---|---|---|---|
| Best Q-learning agent | 7.91 ± 2.44 | 100% | 13.09 | 0.00 |
| Baseline agent | -33.14 ± 83.45 | 80% | 49.94 | 0.00 |
| Optimal (value iteration) | 7.97 ± 2.36 | 100% | 13.03 | 0.00 |

The best agent delivers the passenger in **100%** of episodes, with mean reward **7.91** in 13.09 steps on average and zero illegal actions. That is within 0.06 reward of the optimal policy on the same start states, and its greedy action is optimal in 96.5% of the states it visited during training. The baseline reaches only -33.14 (80% success). Its successful trips are just as short (12.4 steps), but 20% of starts get stuck in a loop until the 200-step limit, which drags the mean down. Raising α from 0.1 to 0.9 fixes this within the same 2000-episode budget. The video `results/taxi_agent.mp4` shows the best agent on 6 new episodes.

## (H) Stochastic environment (action noise)

**Modification.** Taxi is deterministic by default, so I added a controllable noise source as a Gymnasium `ActionWrapper` (`NoisyActionWrapper`). With probability p, the agent's intended action is replaced by a *different* valid action, chosen uniformly from the other 5. The agent is not told this happened. Noise levels p ∈ {0, 0.10, 0.25}. **Settings:** α = 0.1, γ = 0.99, ε-decay = 0.999, 5000 episodes. I deliberately did *not* reuse α = 0.9 here. With noisy transitions, a large step size lets a single unlucky sample (for

example a random illegal pickup costing −10) overwrite a Q-value. A check run of the α = 0.9 configuration under noise reached only -22.7 eval reward at p = 0.10, -33.6 eval reward at p = 0.25. A small α averages over the noise instead. This is the stability trade-off that part D could not show in the deterministic setting.

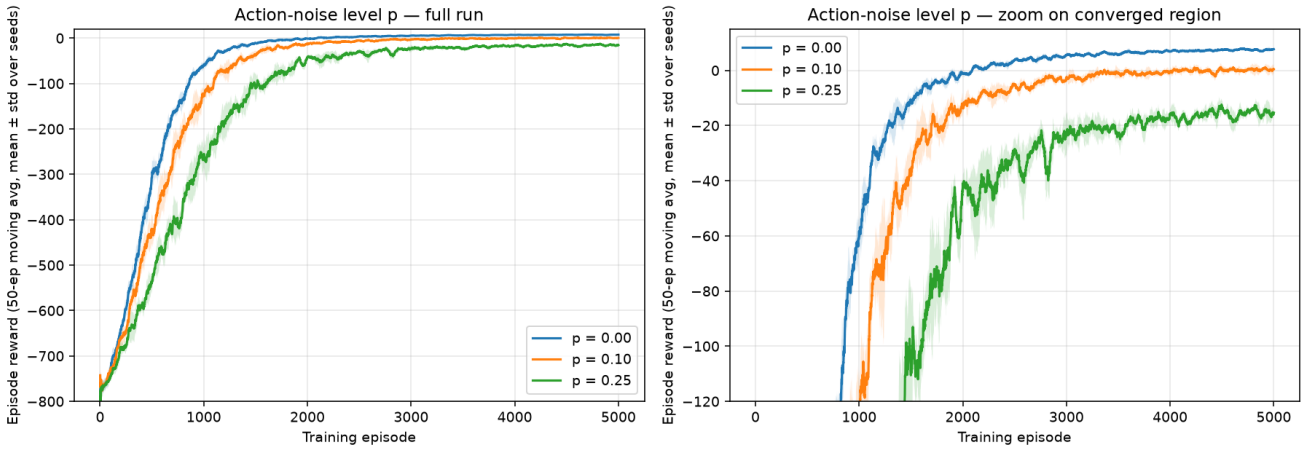



Figure 6 – Training curves under action noise p (each policy is trained and its training reward measured at its own noise level).

| Noise p | Episodes until MA reward ≥ 0 | Mean train reward (all eps) | Train reward last 200 (± std) | Greedy eval reward | Eval success rate | Eval steps |
|---|---|---|---|---|---|---|
| p = 0.00 | 2089 | -70.3 | 7.43 ± 3.35 | 8.22 | 100.0% | 12.8 |
| p = 0.10 | 3364 | -93.8 | 0.33 ± 9.40 | 1.16 | 100.0% | 14.9 |
| p = 0.25 | never | -141.3 | -15.52 ± 19.27 | -14.01 | 100.0% | 19.5 |

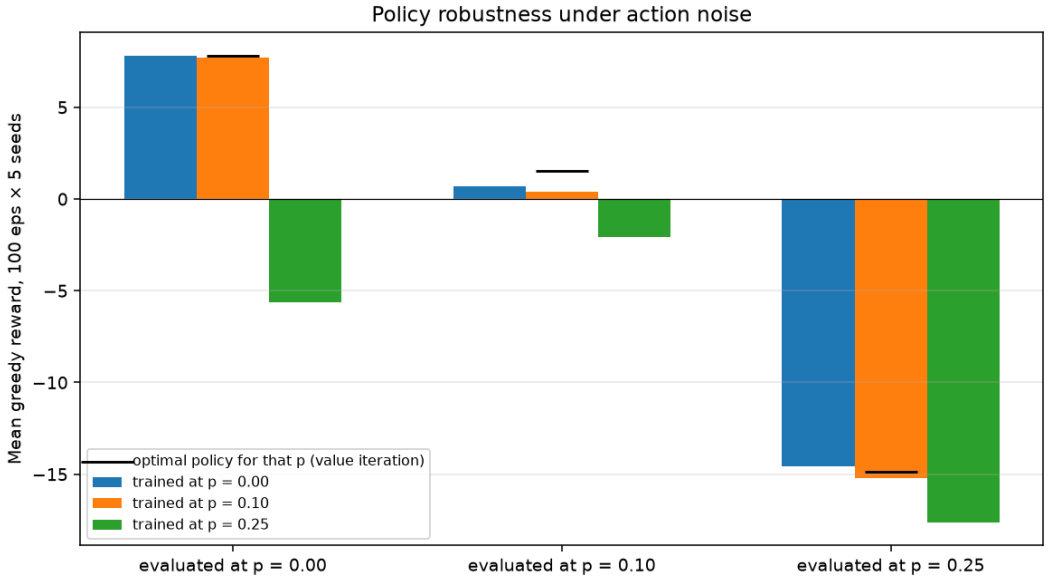



Figure 7 – Cross-evaluation: each trained policy (bar color) evaluated greedily at every noise level (x-axis). Black marks: optimal policy for that noise level (value iteration on the noisy model), i.e. the best any policy can do.

| Trained at \ Evaluated at | p = 0.00 | p = 0.10 | p = 0.25 |
|---|---|---|---|
| p = 0.00 | 7.82 (100%) | 0.70 (100%) | -14.58 (100%) |
| p = 0.10 | 7.73 (100%) | 0.41 (100%) | -15.22 (100%) |

| | | | |
|---|---|---|---|
| p = 0.25 | -5.63 (97%) | -2.07 (100%) | -17.61 (100%) |
| Optimal for that p | 7.82 (100%) | 1.51 (100%) | -14.88 (100%) |

**Learning speed.** Noise slows learning substantially. The moving-average reward reaches 0 after 2089 episodes at p = 0, after 3364 at p = 0.10, and never at p = 0.25. Part of this is real (it is harder to reach the goal), but the TD targets are also noisy, so with a small α many more samples are needed before Q-values settle. The variance across episodes and seeds also grows clearly with p (wider bands in Figure 6).

**Final reward.** Even the *optimal* achievable reward drops from 7.82 (p = 0) to 1.51 (p = 0.10) and -14.88 (p = 0.25). Random substitutions waste steps and sometimes trigger −10 illegal pickups/drop-offs that the agent cannot avoid. The learned policies follow the same trend (8.22 → 1.16 → -14.01 at their own noise level). Success stays at or near 100%, so the agent still delivers the passenger; it just takes longer and pays more penalties.

**Robustness of the original p = 0 policy.** The policy trained without noise remains effective: evaluated at p = 0.10 and 0.25 it gets 0.70 and -14.58, with 100% / 100% success. That is close to the optimal reward for those noise levels (black marks in Figure 7). The reason is that this noise is state-independent: an action that was best without noise is still (almost always) best when every action is equally likely to be corrupted. The noise lowers the achievable return but hardly changes the optimal policy, and a greedy tabular policy simply re-plans from whatever state it ends up in. In this experiment the p = 0 policy actually matches or beats the noise-trained ones (-17.61 for the p = 0.25-trained policy at p = 0.25), because noisy training is less converged after the same 5000 episodes. The p = 0.25-trained policy also drops to -5.63 (97% success) when the noise is removed, since a few of its Q-values are still poor estimates. Overall, uniform action noise mainly costs *sample efficiency* and *achievable reward*, not policy quality. A noise source that changes which action is best (for example wind pushing in one direction) would make noise-aware training more important.

## Conclusions

Tabular Q-learning solves Taxi to optimality within 2000 episodes when the hyperparameters fit the problem. (1) Because the environment is deterministic, a large learning rate (α = 0.5–0.9) is both fast and stable, while α = 0.1 looks converged on the training curve but leaves a ~15% failure rate, and α = 0.01 fails. (2) A far-sighted discount (γ = 0.99) is essential because the only positive reward comes at the end of the trip; γ = 0.5 makes the agent myopic. (3) The exploration schedule trades training reward against state coverage. (4) Once the environment becomes stochastic, the learning-rate trade-off reverses: small α is needed to average out noise, learning slows down, and the achievable reward drops, but the policy learned without noise stays near-optimal under uniform action noise.